

CubeTriage

A decision-driven migration engine
for legacy OLAP cubes

— *in active development* —

One exported file in — a proposed star schema, a quantified
complexity report and a human-review workbook out.
Every trap found, flagged and priced.

*Validated on Adventure Works 2019 Enterprise Edition —
120 review decisions found in ~1.5 seconds*

Malin Jacobsson

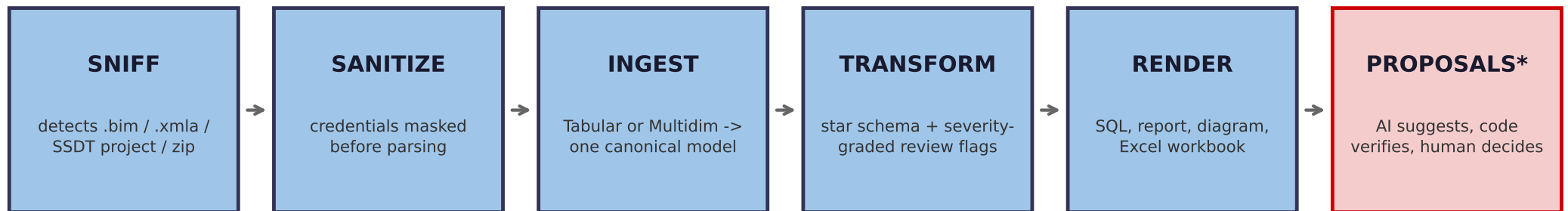
The problem

Migrating a legacy SSAS cube to a modern relational model costs a senior consultant weeks: untangling MDX, mapping role-playing dimensions, finding the KPIs and security roles nobody documented.

Before anyone can even price the migration, somebody spends days just answering: what is actually IN this cube?

CubeTriage answers that question in seconds — from a single exported metadata file. No cube data is read.

How it works



The first five stages are deterministic — same input, same output.
Proposals are opt-in behind a backend interface (mock / local LLM).

Outputs: *_canonical.json · *_out.json · *_schema.sql · *_load.sql ·
*_validation.sql · *_review.md · *_hitl.xlsx · *_diagram.png ·
*_decisions.json · *_run_report.json

Result on a real cube

Adventure Works 2019 Enterprise Edition (Multidimensional) —
Microsoft's own reference cube, built to demonstrate every
enterprise SSAS feature.

16 dimensions · 11 facts · 79 relationships · 38 measures

mapped automatically

22 calculated measures in MDX

flagged for manual translation — not ignored

29 role-playing · 4 parent-child · 4 reference · 6 m2m bridges

structural traps found

31 unmigratable constructs (KPIs, actions, perspectives)

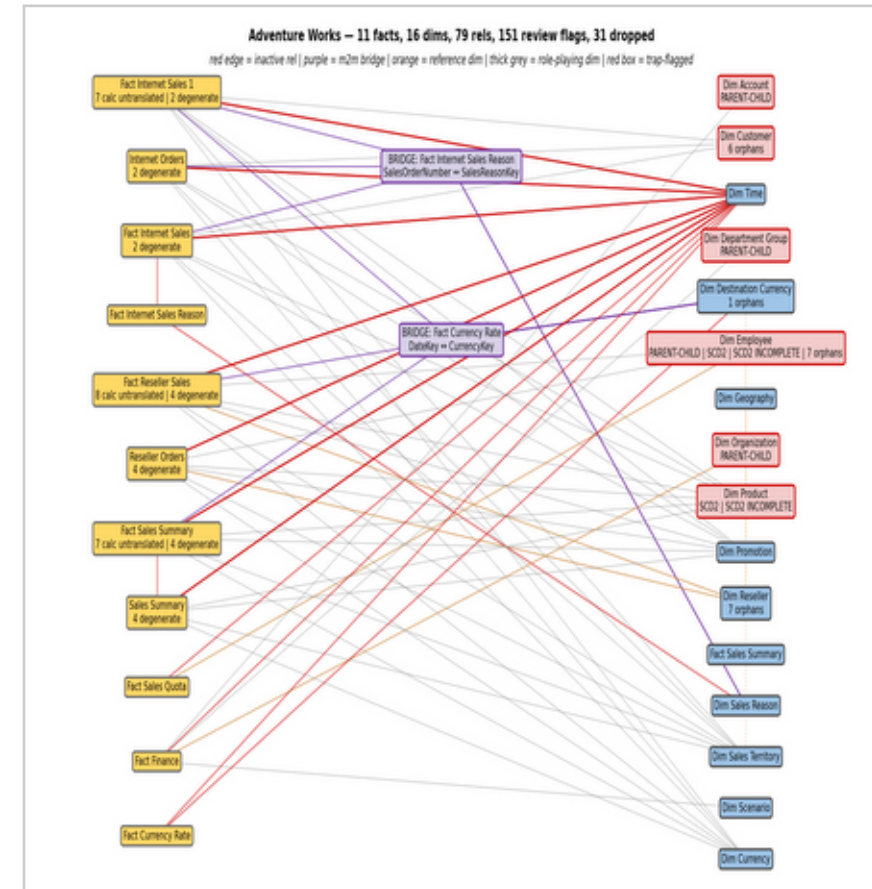
reported, never lost

101 inferred assumptions

each documented in its own channel

120 human-review items

the workbook IS the work order



Full pipeline run: ~1.5 seconds. Also verified on the
Tabular AdventureWorks .bim (compat 1400) — tables classified
purely from the relationship graph, no naming conventions needed.
Newest: generated load SQL + a 101-check validation harness.

The review loop

Reviewing IS migrating — each decision mutates the model and regenerates the artifacts, not just the report.

Every warn flag and dropped item becomes a numbered decision

decisions.json — stable ids,
survives re-runs

Click a flag — see where the problem lives

canonical source slice +
star-schema target table

Buttons say the outcome, not the process

Keep column · Drop column · Provide
SQL — decided entries persist

Zero-dependency, offline, runs where the data lives

the Excel workbook stays as
the exportable record

*First run ~1.5 s — then each decision cycle
(apply + regenerate all artifacts) costs ~1 second.*

The screenshot displays the DMC886F - scd2_dimension decision review interface. At the top, a summary bar shows '14 of 128 decided - 84 flags - 5 assumptions - 11 dropped'. Below this, a table lists various source and target tables with their status (warn, error, or open) and a 'warn' flag. The 'warn' flag is highlighted in yellow. The main panel shows the decision details for 'scd2_dimension'. It includes a 'Type-2 dimension - Fact FK joins surrogate EmployeeKey; point-in-time vs current-version (I + SimpleStartDate/SimpleEndDate) must be decided' description. The 'SOURCE' section shows the 'MODEL/SCD2/DIMENSION/EMPLOYEE' table with its schema. The 'TARGET' section shows the 'dim_employee' table. The 'DECISION - CURRENT: OPEN' section has buttons for 'Keep SCD' and 'Custom override...'. A 'Comment...' field and 'Save comment' and 'Reopen' buttons are also present.

The star — drawn live

Power BI-style model view

table cards with real columns,
FK edges anchored at the key rows

*** -> 1 cardinality, dashed
bridges for many-to-many**

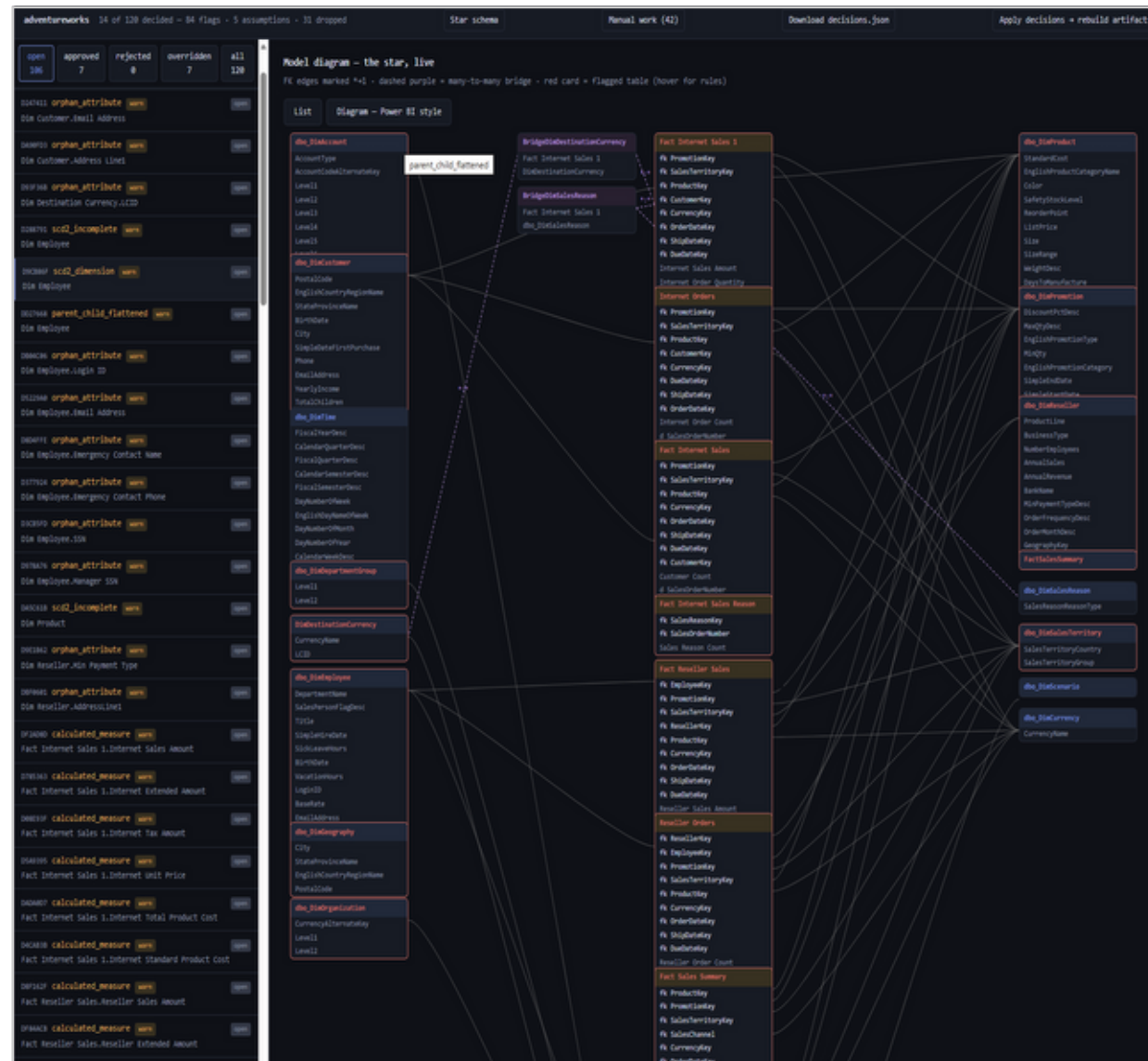
role-playing dims fan out
exactly like the source model

Flagged tables tinted red

hover shows which rules hit —
the map doubles as a risk radar

Rebuilt after every decision

dropped columns vanish, provided
SQL appears — watch it grow



What's left for a human

The Manual work view is the honest backlog — everything the engine can't produce itself, counted live in the header.

Calculated measures need SQL

each row opens its decision —
Provide SQL writes it into the model

Artifact notes to verify

guessed source tables and
unresolved bridges, never hidden

The counter is the workload

empty queue + empty manual work
= a finished, reproducible spec

The screenshot displays the 'Manual work' view in a data engineering tool. The top header shows the status of the 'adventureworks' model: 14 of 110 decided, 84 flags, 5 assumptions, and 31 dropped. The 'Manual work (41)' tab is active, showing a list of artifacts that need SQL. The left sidebar lists artifacts with their status (open, approved, rejected, overridden, all) and a count. The main panel shows the details of the selected artifact, 'Fact Internet Sales 1.Internet Sales Amount', with a description and a list of related artifacts.

open	approved	rejected	overridden	all
106	7	0	7	120

adventureworks 14 of 110 decided - 84 flags - 5 assumptions - 31 dropped

Star schema Manual work (41) Download decisions.json Apply decisions + rebuild artifacts

Manual work - what's left for a human

Everything the engine can't produce itself lands here. Write the SQL via the decision (Provide SQL) or drop the item - then rebuild.

CALCULATED MEASURES NEEDING SQL (21)

Fact Internet Sales 1.Internet Sales Amount

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Extended Amount

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Tax Amount

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Freight Cost

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Unit Price

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Total Product Cost

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Standard Product Cost

write the SQL expression (or drop the measure) + open decision

Fact Reseller Sales.Reseller Sales Amount

write the SQL expression (or drop the measure) + open decision

Fact Reseller Sales.Reseller Extended Amount

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Sales Amount

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Extended Amount

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Tax Amount

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Freight Cost

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Unit Price

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Total Product Cost

write the SQL expression (or drop the measure) + open decision

Fact Internet Sales 1.Internet Standard Product Cost

write the SQL expression (or drop the measure) + open decision

Fact Reseller Sales.Reseller Sales Amount

write the SQL expression (or drop the measure) + open decision

Fact Reseller Sales.Reseller Extended Amount

write the SQL expression (or drop the measure) + open decision

Design principles

Nothing is hidden

Constructs the engine sees but cannot migrate are recorded in a dropped channel. A reviewer always sees the full picture.

Provenance is structural

Observed facts, inferred values, dropped constructs and AI proposals are four separate channels — what the engine KNOWS vs GUESSES.

AI proposes — never decides

An optional backend may suggest resolutions. Code verifiers check every proposal against the model; the human approves.

Runs where the data lives

Deterministic, offline-capable, no telemetry — designed to run as a container inside the customer's own perimeter.